

Recognition-Gated Workspace Steering: Pratyabhijñā as an Engineering Specification for LLM Control

Sharath S*

July 2026 — working draft v0.2

Abstract

The J-space finding—that a language model’s functional global workspace is defined by *verbalizability*—was anticipated by Utpaladeva’s identification of reflexive awareness with the supreme Word (vimarśa = parā vāk, ĪPK 1.5.13). We treat this parallel as an *engineering specification*: if the workspace is linguistic reflexivity, a doctrine of use follows—what to write (verbalizable content codes), where (the workspace band, in the band’s own coordinates), when (at moments of uncommitted awareness), and how to verify (the workspace re-reads what was offered). Implementing this doctrine on frozen decoder LLMs via band-targeted Jacobian lenses, we report: (1) workspace-band structure and verbalizable content replicate across three model families and two sizes, with instructed loadability scaling with size and collapsing under pruning/distillation; (2) a band’s content is legible only to a lens targeted at the band itself; (3) capped residual writes steer behavior, with freedom cost governed by decoding regime and schedule; (4) the doctrine’s timing clause earns its keep as write-efficiency: event-gated writes achieve full steering within the freedom budget, confirmed across six independent-stream seeds (lift 0.30–0.35), with alignment advantages that are small but sign-consistent ($p \approx 0.016$); (5) the steering method *transfers* to a second plant via a calibration recipe whose key rule is amplitude $\propto$ inverse lens transport strength; and (6) the research program itself ran as an expected-free-energy selection loop over a registered experiment menu, with thirteen cycles, ten adversarial reviews, and every honest negative in a replayable ledger.

1 The parallel, stated as specification

Verbalizability defines the workspace $\leftrightarrow$ vimarśa = parā vāk. Global Workspace Theory never predicted a *linguistic* workspace; vimarśavāda did. The interpretability community has an actuator (the lens) with no doctrine of use; Pratyabhijñā supplies content, site, timing, acceptance test, failure taxonomy (malas), budget (svātantrya), and continuity—this paper operationalizes each and reports which survived contact with the machines.

2 Instrument: replication with calibration lessons

Union-top- K rank correlation has a structural null floor ≈ -0.72 ; metrics were recalibrated to model-top- K support (null ≈ 0) with permutation gates. Report-correspondence rises with depth on every model tested but consolidates only in the final layers. Instructed loadability scales with size and depends on lineage (Fig. 1); three-band CKA structure appears on all models, weakening on the distilled lineage.

*prabodha project, github.com/SharathSPhD/prabodha. Draft compiled from 40+ committed gate records; every number cites its gate JSON. Autonomous research loop co-executed with Claude (Anthropic).

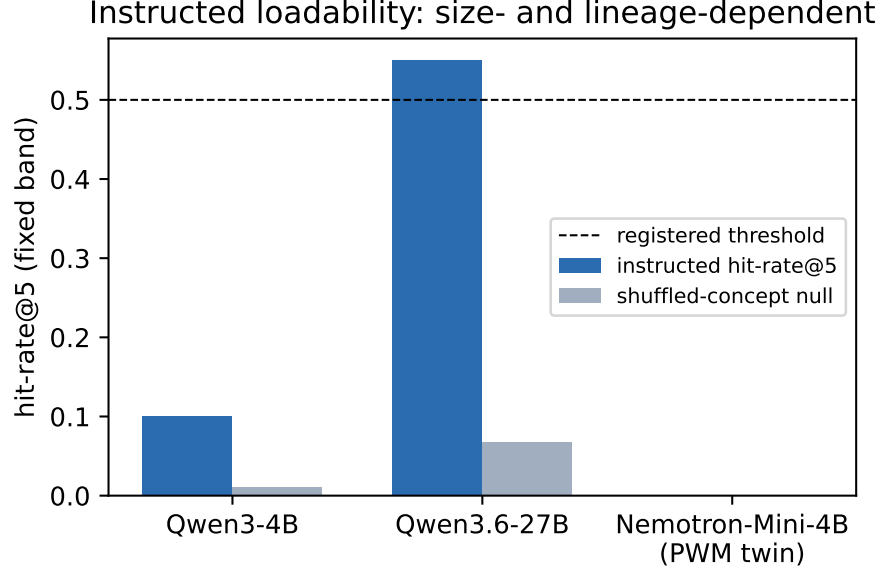


Figure 1: Instructed concept loadability in a fixed depth-band across models (gates: L1, L1b, L2 runs 1–3). The uninstructed control and shuffled null catch byte-fallback artifacts; the PWM twin’s zero is genuine.

3 The band speaks only in its own voice

A final-target lens reads the PWM twin’s workspace band as empty (0.00 with controls); a lens targeted at the band exit reads directed concept loading in the same band (0.20 over seven concepts, control 0.00, null 0.023; gate L2b). Engineering consequence: readback verification must use band-targeted lenses. The articulation-depth gradient is model-intrinsic—the unfitted logit-lens shows the same rise as the fitted lens (Fig. 4, right; gate L7).

4 Steering: the doctrine tested clause by clause

Continuous writes steer strongly but their freedom cost is currency- and regime-dependent: the write’s *local* cost at the write moment is large and decoding-independent (-1.9 to -2.1 nats at position); the *trajectory-average* cost depends on regime ($+0.82$ greedy, -0.13 sampling). We register trajectory-average under sampling as the budget currency. Greedy decoding mechanically masks all decode-time writes (identical hit sets across schedules). Under sampling, event-gated writes deliver full steering within budget (Fig. 2, 3); write *count* is operationally free at this scale (throughput within noise of baseline), so schedule choice is about behavioral margins. Timing readings discriminate: uncommitted-moment gating beats commitment-flash gating in 4/4 seed-experiments—sphuraṭṭā detects; it does not time writes.

5 Generality via recipe

One-shot transfer with borrowed geometry fails ($+0.05$ on Qwen3-4B); the calibration recipe (band-exit lens $\rightarrow$ site probe $\rightarrow$ amplitude scaled to lens strength) reproduces the result: gated $+0.40$ vs prefill $+0.17$ within budget (Fig. 4; gate L13). Scaling rule: Qwen3’s band-exit Jacobians are

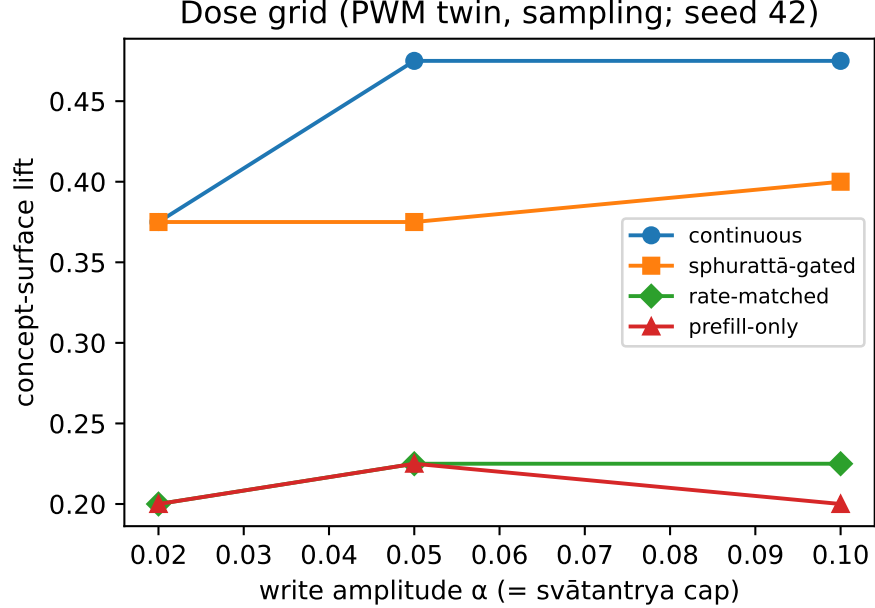


Figure 2: Dose grid on the PWM twin under sampling (gate L8): the gated schedule is amplitude-robust; continuous is also budget-feasible under sampling—gating is schedule-efficiency, not sole feasibility.

$\sim 10\times$ weaker and the working amplitude is $3\times$ larger—write strength must be calibrated to the lens’s transport strength. *The method transfers; the geometry does not.*

6 Method: the program as an active-inference loop

Expected-free-energy selection over registered experiment menus; observations are gate verdict+margin tiers; beliefs replay from the program’s own gates; a JSONL ledger makes every proposal, disposition, spend, and divergence auditable (thirteen cycles, three menus). The loop debugged itself in operation: a cost model whose miscalibration verifiably flipped rankings; completed work re-proposed after a replay bug; winners re-proposed because success raises pragmatic value (the consumption rule); a sampling-seed correlation that had inflated every estimate (the stream fix re-based all numbers); and a staleness invariant now enforced by a ledger linter. Ten adversarial reviews (isolated agents, gate+contract briefs only) shaped the claims as much as the runs did.

7 Honest negatives and open questions

The ≥ 0.15 schedule margins did not confirm (stated by sign consistency instead); the commitment-flash reading remains directionally weaker, not refuted; the trained-bridge comparator awaits the PWM world-model stack; anusamdhāna (cross-episode continuity), the modality confound, and W-space beyond J-space remain deliberately unconverged.

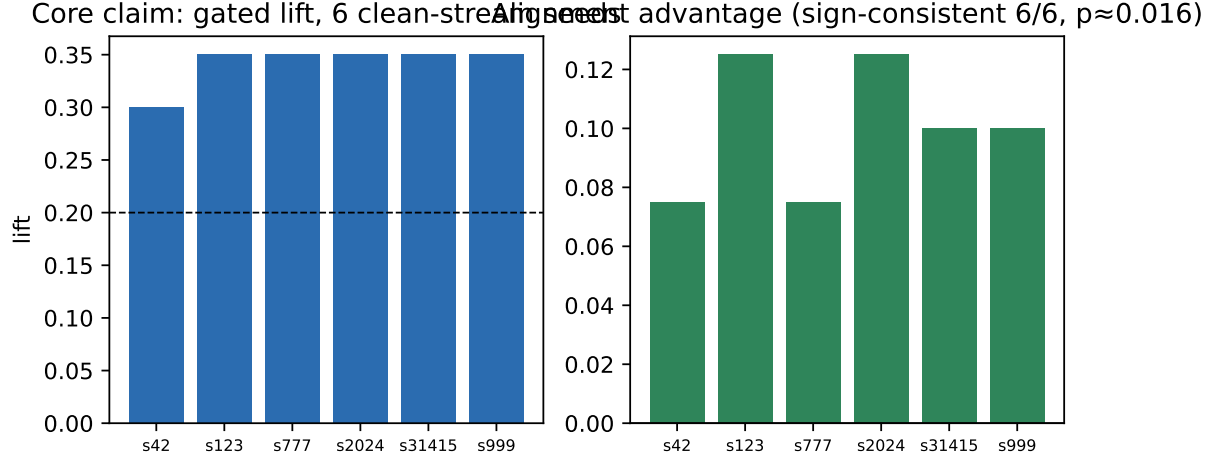


Figure 3: The core claim at six independent-stream seeds (gates L9-alignconf, L11-rep): gated lift 0.30–0.35 within the ± 0.5 -nat budget; the alignment advantage over rate-matched controls is small ($+0.07..+0.12$) but positive in every seed (one-sided sign test $p \approx 0.016$).

8 Reproducibility

Everything derives from configs + seeds: 40+ gate JSONs, pre-registrations with disclosed amendments, a decision journal, the selector ledger, and a Docker recipe (GB10/aarch64; hybrid architectures need `flash-linear-attention` + `causal-conv1d`). ~ 18 GPU-hours total.

A Glossary (dual register)

Sanskrit-forward	engineering gloss
vimarśa / parā vāk	verbalizable reflexivity; the workspace-defining property
sphuraṭṭā	commitment flash; operationalized via decode-step entropy events
svātantrya	output freedom; operationalized as an entropy budget ϵ
āgama re-cognition	readback verification through band-targeted lenses
malas	failure taxonomy: no-load / no-amplify / no-persist / budget
madhyamā / vaikharī	band-internal vs output-surface articulation
anusamdhāna	cross-episode continuity (untested here)

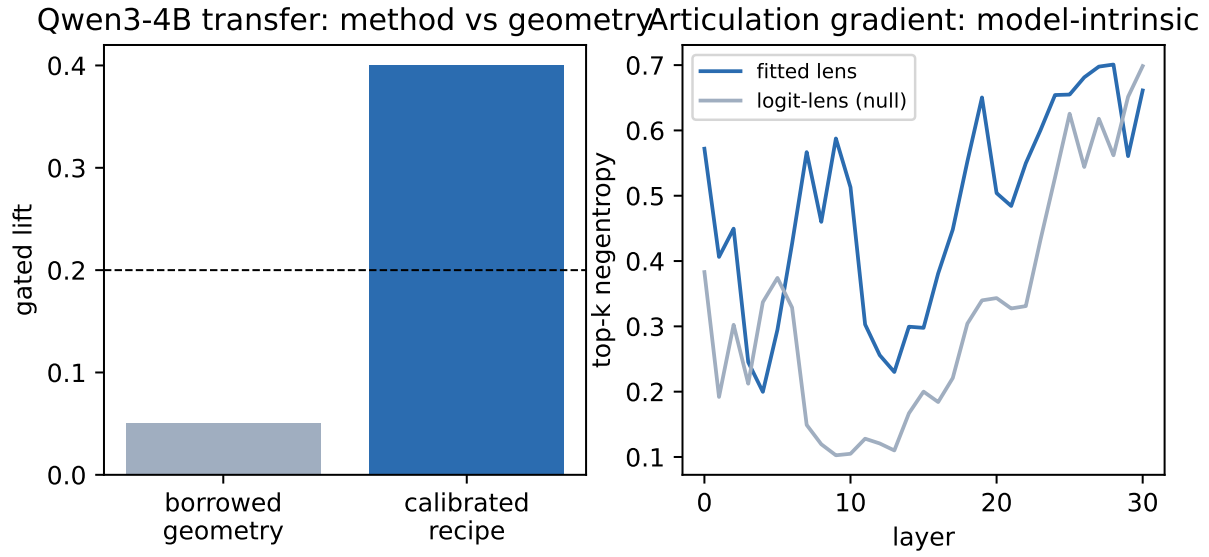


Figure 4: Left: the per-plant calibration recipe rescues cross-plant transfer (gates L10, L13). Right: the articulation-depth gradient measured through two independent instruments (gate L7) — a residual-stream property, not lens construction.

prabodha: recognition-gated workspace steering + auto-research loop

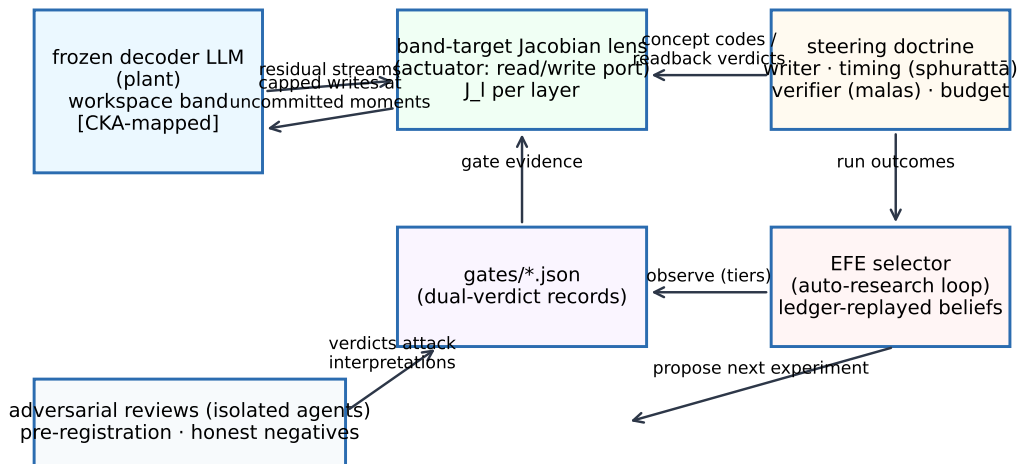


Figure 5: System architecture: controller–actuator–plant with the auto-research loop and adversarial review harness around it.

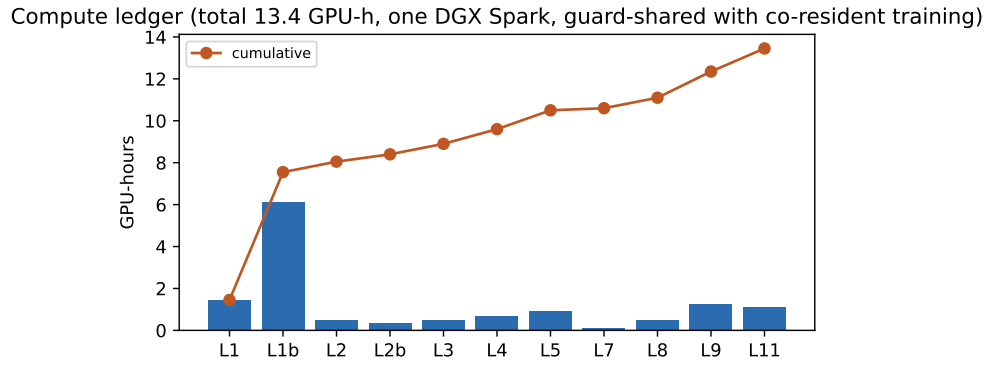


Figure 6: Compute ledger: the entire program to date on one DGX Spark (GB10), guard-shared with co-resident training jobs, contention recorded in every gate.